

ИДЗ №2 по ОС.

Магомедов Абдул Омаргаджиевич. БПИ 234.

Вариант ИДЗ: 15 вариант.

Условие:

Задача о больнице. В больнице два дежурных врача принимают пациентов, выслушивают их жалобы и отправляют или к стоматологу, или к хирургу, или к терапевту. Стоматолог, хирург и терапевт лечат пациентов. Каждый врач может принять только одного пациента за раз. Пациенты стоят в очереди к врачам и никогда их не покидают.

Создать много процессное приложение, моделирующее рабочий день клиники.

Каждого из врачей и пациентов моделировать отдельными процессами.

Программа на оценку 4-5:

Сценарий решаемой задачи:

Программа моделирует рабочий день клиники.

Родительский процесс создаёт:

- Два дочерних процесса — **регистраторы** (дежурные врачи), которые "принимают" пациентов и направляют их к нужным врачам;
- Три дочерних процесса — **врачи** (стоматолог, хирург, терапевт), которые "лечат" пациентов;
- Пациенты моделируются как сущности внутри регистраторов, у каждого пациента есть свой номер и назначение к врачу.

Регистраторы генерируют пациентов, присваивают им уникальный номер и специальность (случайным образом), после чего добавляют пациента в общую очередь.

Врачи получают пациентов из очереди и принимают их **в порядке регистрации**, строго по своей специальности.

В программе реализована синхронизация доступа к очереди пациентов и учёт количества вылеченных пациентов. Программа завершается только после того, как **все пациенты прошли лечение**. Время обслуживания врачом задается случайно от 1 до 3 секунд с помощью:

```
int duration = rand() % 3 + 1;
```

Для лучшей работы программы был установлен лимит пациентов, программа проверяет введенное пользователем кол-во пациентов (1-100).

```
arbuz@AsusVivobook:~/clinic_6_7$ ./clinic_6_7 0  
Некорректное количество пациентов (1 - 100)
```

```
arbuz@AsusVivobook:~/clinic_6_7$ ./clinic_6_7  
Использование: ./clinic_6_7 <кол-во_пациентов>
```

Дополнительно в программе реализован **неименованный POSIX семафор** `print_mutex`, который используется исключительно для синхронизации вывода сообщений в консоль. Это позволяет устранить визуальные несоответствия в порядке отображения информации (например, когда пациент №2 выводится раньше пациента №1).

Семафор `print_mutex` предотвращает одновременный доступ к потоку `stdout` несколькими процессами, обеспечивая правильный порядок сообщений в консоли при запуске программы.

Взаимодействие процессов и используемые механизмы:

В программе используется схема:

b) Множество процессов взаимодействуют с использованием неименованных POSIX семафоров. Обмен данными ведется через разделяемую память в стандарте POSIX.

- **Разделяемая память** (`mmap`) используется для хранения очереди пациентов и состояния (фронт/тыл очереди, номера, счётчики).
- **Семафоры:**
 - `queue_mutex` — для доступа к очереди пациентов;
 - `patient_ready` — сигнализирует врачам о наличии пациентов;

- doctor_sem[N] — врачи принимают только по одному пациенту одновременно;
- treated_mutex — защита счётчика вылеченных пациентов.

Завершение программы:

Программа завершается корректно, когда:

- Все пациенты были направлены к врачам;
- Все пациенты были вылечены;
- После этого родительский процесс завершает процессы врачей через kill.

Пример завершения программы можно посмотреть далее в пункте **Результат работы программы**.

Также предусмотрено завершение по сигналу SIGINT (Ctrl+C). В этом случае вызывается обработчик сигнала, который освобождает ресурсы и завершает выполнение.

```
arbuzz@AsusVivobook:~/clinic_4_5$ ./clinic_4_5 3
[Регистратор 2] Пациент #1 направлен к врачу: Стоматолог
[Стоматолог] Приём пациента #1...
^C
Программа завершена по сигналу.
```

Очистка ресурсов:

В программе реализовано:

- Удаление разделяемой памяти через munmap;
- Уничтожение всех семафоров через sem_destroy;
- Обработка SIGINT через signal(SIGINT, ...) с вызовом cleanup().

Очистка выполняется как при штатном завершении, так и при аварийном завершении с клавиатуры.

Результат работы программы:

Программа работает стабильно при любом количестве пациентов (например: ./clinic_4_5 10). В консоли отображается:

- Какой регистратор направил какого пациента к какому врачу;
- Какой врач начал приём какого пациента;
- Когда пациент был выписан.
- Время лечения каждого пациента

```
arbuzz@AsusVivobook:~/clinic_4_5$ ./clinic_4_5 3
[Регистратор 1] Пациент #1 направлен к врачу: Хирург
[Регистратор 2] Пациент #2 направлен к врачу: Терапевт
[Хирург] Приём пациента #1...
[Терапевт] Приём пациента #2...
[Регистратор 2] Пациент #3 направлен к врачу: Терапевт
[Хирург] Пациент #1 выписан.
[Терапевт] Пациент #2 выписан.
[Терапевт] Приём пациента #3...
[Терапевт] Пациент #3 выписан.

Клиника завершает работу. Завершаем процессы врачей...
```

Результаты подтверждают корректность моделирования:

- Все пациенты лечатся строго по очереди;
- Каждый врач обслуживает только свою специализацию;
- Программа завершает работу только после лечения всех пациентов.

Программа на оценку 6-7:

Взаимодействие процессов и используемые механизмы:

В данной версии применяется схема:

а) Множество процессов взаимодействуют с использованием **именованных POSIX семафоров**. Обмен данными ведется через **разделяемую память в стандарте POSIX**.

Семафоры:

- /queue_mutex — синхронизация доступа к очереди;
- /treated_mutex — защита счётчика вылеченных пациентов;
- /patient_ready — сигнал для врачей, что есть пациенты;
- /doctor_0, /doctor_1, /doctor_2 — по одному семафору на каждого врача.

Отличие от версии 4–5:

- Используются **именованные POSIX семафоры** через sem_open, sem_close, sem_unlink.
- В версии на 4–5 использовались **неименованные семафоры** через sem_init, sem_destroy.

Очистка ресурсов:

Помимо освобождения разделяемой памяти через munmap, в этой версии:

- Все **именованные семафоры закрываются** через sem_close.
- Дополнительно **удаляются из системы** через sem_unlink("/имя").

Таким образом, в отличие от версии 4–5, здесь требуется удаление имён семафоров, чтобы избежать конфликтов при повторном запуске программы.

Все остальные пункты:

Идентично отчёту для программы на оценку 4–5.

Программа на оценку 8:

Для завершения приложения используются те же подходы, что и в предыдущих решениях:

В данной реализации, как и в предыдущих на оценки 4–5 и 6–7, завершение осуществляется при помощи контроллера. Контроллер отслеживает количество обслуженных пациентов, и после того, как последний пациент выписан, приложение завершает работу.

При этом выводится сообщение:

«Клиника завершает работу. Все пациенты вылечены. Нажмите Ctrl+C для выхода.»

Процесс controller обрабатывает сигнал SIGINT (Ctrl+C) и корректно очищает ресурсы:

- разделяемую память (через shmctl)
- семафоры (через semctl)

Приложение использует обмен данными через разделяемую память. Применение семафоров реализовано следующим образом:

В данной версии приложения:

- Обмен данными между независимыми процессами реализован через **разделяемую память** на основе **UNIX System V shared memory** (shmget, shmat, shmdt, shmctl)
- Для синхронизации процессов используются **UNIX System V семафоры** (semget, semop, semctl)
- Объявлено несколько семафоров:
 - SEM_MUTEX — для защиты общей очереди (вставка/извлечение пациентов)
 - SEM_READY — для счётчика новых пациентов в очереди
 - SEM_TREATED — защита счётчика обслуженных пациентов
 - SEM_DOCTOR_BASE + i — блокировка врачей по специальности

Каждый компонент (врач, регистратор, контроллер) является независимой программой и взаимодействует с остальными исключительно через общие структуры данных и семафоры.

В отчёт добавлена информация о новой реализации и приведены соответствующие результаты работы программы:

Добавленная реализация:

- В отличие от предыдущих решений, программа состоит из **независимых исполняемых файлов**:
 - controller — запускается первым, инициализирует ресурсы
 - registrar — запускается с аргументом ID (1 или 2), направляет пациентов
 - doctor — запускается с аргументом специальности (0 – стоматолог, 1 – хирург, 2 – терапевт)
- Все процессы могут быть запущены:
 - либо вручную из отдельных терминалов
 - либо автоматически через предоставленный скрипт run.sh

```
arbuzz@AsusVivobook:~/clinic_8$ [Регистратор 2] Пациент #1 направлен к врачу: Стоматолог
[Регистратор 1] Пациент #2 направлен к врачу: Терапевт
[Стоматолог] Приём пациента #1 (3 сек)
[Терапевт] Приём пациента #2 (1 сек)
[Регистратор 2] Пациент #3 направлен к врачу: Терапевт
[Регистратор 1] Пациент #4 направлен к врачу: Стоматолог
[Терапевт] Пациент #2 выписан
[Терапевт] Приём пациента #3 (3 сек)
[Регистратор 2] Пациент #5 направлен к врачу: Хирург
[Стоматолог] Пациент #1 выписан
[Регистратор 1] Пациент #6 направлен к врачу: Стоматолог
[Стоматолог] Приём пациента #4 (2 сек)
[Хирург] Приём пациента #5 (1 сек)
[Хирург] Пациент #5 выписан
[Терапевт] Пациент #3 выписан
[Стоматолог] Пациент #4 выписан
[Стоматолог] Приём пациента #6 (3 сек)
[Стоматолог] Пациент #6 выписан

Клиника завершает работу. Все пациенты вылечены. Нажмите Ctrl+C для выхода.
```

Так же важно учесть, чтобы исполняемый файл run.sh работал корректно, необходимо прописать **chmod +x run.sh**, и далее, к примеру ./run.sh 6.

Программа на оценку 9:

Завершение приложения:

Завершение всех процессов организовано корректно:

- Контроллер (controller9) отслеживает, сколько пациентов вылечено, и завершает работу автоматически.
- При завершении:
 - В очереди врачей отправляется специальное сообщение с номером пациента 0, означающее завершение.
 - С помощью SIGTERM через system("pkill ...") завершаются все doctor9 и registrar9.
 - Весь вывод синхронизирован через sem_print.
- Очистка всех ресурсов (mq_unlink, sem_unlink, shm_unlink) производится через cleanup() — вызывается и при SIGINT, и при завершении штатно.

Дополнительно:

- Врачи обрабатывают сигнал SIGTERM и завершаются чисто.
- Скрипт run.sh ловит Ctrl+C и завершает все процессы.

Используемые средства взаимодействия:

Вариант:

Множество независимых процессов взаимодействуют с использованием именованных POSIX семафоров. Обмен данными ведётся через очереди сообщений в стандарте POSIX.

Использовано:

- Очереди сообщений POSIX:
 - /q_d — стоматолог
 - /q_s — хирург
 - /q_t — терапевт

- Семафоры POSIX (sem_open):
 - /treated_sem — сигнал врачу → контроллеру о вылеченном пациенте
 - /print_mutex — мьютекс для синхронизации вывода в консоль
 - /counter_mutex — мьютекс на счётчик
- Разделяемая память POSIX:
 - /clinic_counter — счётчик номеров пациентов (гарантирует уникальность)

Добавленная реализация и финальный результат:

Структура:

- Приложение состоит из 6 независимых процессов:
 - 1 контроллер
 - 2 регистратора
 - 3 врача (по одному на специализацию)
- Все они запускаются из run.sh:

```

arbuz@AsusVivobook:~/clinic_9$ ./run.sh 3
[Регистратор 2] Пациент #1 направлен к врачу: Стоматолог
[Хирург] Приём пациента #2 (2 сек)...
[Стоматолог] Приём пациента #1 (1 сек)...
[Регистратор 1] Пациент #2 направлен к врачу: Хирург
[Регистратор 2] Пациент #3 направлен к врачу: Хирург
[Стоматолог] Пациент #1 выписан.
[Хирург] Пациент #2 выписан.
[Хирург] Приём пациента #3 (1 сек)...
[Хирург] Пациент #3 выписан.

Клиника завершает работу. Все пациенты вылечены.

Ресурсы удалены

Все процессы остановлены

```

Количество пациентов делится между регистраторами автоматически.

Поведение:

- Каждый пациент получает уникальный номер через защищённую разделяемую память.
- Регистрируется случайным врачом.
- Врачи читают сообщения из очередей и обрабатывают пациентов.
- После окончания лечения врачи сигнализируют об этом контроллеру через семафор.
- После приёма всех пациентов врачи получают спец-сообщение (номер == 0) — и завершаются.
- Скрипт отслеживает завершение и при Ctrl+C завершает всё.

Итоги:

- Сообщения не теряются.
- Уникальность номеров соблюдена.
- Нет дублирующего вывода.
- Все ресурсы очищаются.
- Программа завершает работу как при естественном окончании, так и по Ctrl+C.

Программа на оценку 10:

Завершение приложения:

Приложение завершает работу корректно и безопасно:

- Контроллер (controller10) отслеживает количество вылеченных пациентов через семафор SEM_TREATED.
- После того как все пациенты вылечены:
 - Контроллер отправляет в очереди специальное сообщение (num = 0) — сигнал к завершению.
 - Через system("pkill -TERM ...") отправляются сигналы врачам и регистраторам.
 - Процессы врачей и регистраторов завершаются через обработчики SIGTERM.
- Все IPC-ресурсы (msgget, semget, shmget) очищаются:
 - Очереди (msgctl IPC_RMID)
 - Семафоры (semctl IPC_RMID)
 - Разделяемая память (shmdt, shmctl IPC_RMID)

Всё работает как при штатном завершении, так и при Ctrl+C.

Используемый способ взаимодействия:

Вариант взаимодействия:

Множество независимых процессов взаимодействуют с использованием **семафоров в стандарте UNIX System V**.

Обмен данными ведётся через **очереди сообщений UNIX System V**.

Использовано:

- **Очереди сообщений System V:**
 - MSG_D — стоматолог
 - MSG_S — хирург
 - MSG_T — терапевт
- **Семафоры System V (один набор SEM_KEY, три индекса):**
 - SEM_PRINT — синхронизация консоли
 - SEM_COUNTER — защита счётчика пациентов

- SEM_TREATED — сигнал от врача → контроллеру о вылеченном пациенте
- **Разделяемая память (SHM_KEY):**
 - Счётчик уникальных номеров пациентов.

Добавленная реализация и поведение:

Процессы:

- controller10: контролирует и завершает работу, очищает ресурсы.
- registrar10: создаёт пациентов с уникальными номерами, направляет к специалистам.
- doctor10: обслуживает только свою очередь, моделирует лечение, отчитывается через семафор.

Запуск:

Файл run.sh позволяет:

- Компилировать всё автоматически.
- Запустить всех участников, в том числе:
 - 2 регистратора (деление по количеству пациентов)
 - 3 врача (по специализациям)
 - контроллер

Также ловит Ctrl+C и завершает всё через pkill -TERM.

Поведение:

- Каждому пациенту выдаётся уникальный номер (через shmget + semop).
- Пациенты обрабатываются врачами по специализации.
- Завершение происходит корректно, консольный вывод не пересекается.